



**TOP 50
PANDAS
INTERVIEW QUESTIONS**

Part : 1

Q-1

1757. Recyclable and Low Fat Products

Solved 

Easy

 Topics

 Companies

[SQL Schema >](#) [Pandas Schema >](#)

Table: `Products`

Column Name	Type
product_id	int
low_fats	enum
recyclable	enum

product_id is the primary key (column with unique values) for this table.

low_fats is an ENUM (category) of type ('Y', 'N') where 'Y' means this product is low fat and 'N' means it is not.

recyclable is an ENUM (category) of types ('Y', 'N') where 'Y' means this product is recyclable and 'N' means it is not.

pratik

Write a solution to find the ids of products that are both low fat and recyclable.

Return the result table in **any order**.

The result format is in the following example.

Example 1:

Input:

Products table:

product_id	low_fats	recyclable
0	Y	N
1	Y	Y
2	N	Y
3	Y	Y
4	N	N

Output:

product_id
1
3

Explanation: Only products 1 and 3 are both low fat and recyclable.

Solution :

Pandas  Auto

    

```
1 import pandas as pd
2
3 def find_products(products: pd.DataFrame) -> pd.DataFrame:
4     var = (products['low_fats'] == 'Y') & (products['recyclable'] == 'Y')
5     return products[var][['product_id']]
```

Q-2

584. Find Customer Referee

Solved ✓

Easy Topics Companies Hint

[SQL Schema](#) > [Pandas Schema](#) >

Table: Customer

Column Name	Type
id	int
name	varchar
referee_id	int

In SQL, id is the primary key column for this table.

Each row of this table indicates the id of a customer, their name, and the id of the customer who referred them.

Find the names of the customer that are either:

1. **referred by** any customer with `id != 2`.
2. **not referred by** any customer.

Return the result table in **any order**.

Example 1:

Input:

Customer table:

id	name	referee_id
1	Will	null
2	Jane	null
3	Alex	2
4	Bill	null
5	Zack	1
6	Mark	2

Output:

name
Will
Jane
Bill
Zack

Solution :

</> Code

Pandas ▾ 🔒 Auto

```
1 import pandas as pd
2
3 def find_customer_referee(customer: pd.DataFrame) -> pd.DataFrame:
4     var = (customer['referee_id'] != 2 ) | (customer['referee_id'].isnull() )
5     return customer[var][['name']]
6
```

Q-3:

595. Big Countries

Easy Topics Companies

[SQL Schema](#) > [Pandas Schema](#) >

Table: World

Column Name	Type
name	varchar
continent	varchar
area	int
population	int
gdp	bigint

name is the primary key (column with unique values) for this table.

Each row of this table gives information about the name of a country, the continent to which it belongs, its area, the population, and its GDP value.

A country is **big** if:

- it has an area of at least three million (i.e., 3000000 km²), or
- it has a population of at least twenty-five million (i.e., 25000000).

Write a solution to find the name, population, and area of the **big countries**.

Return the result table in **any order**.

Example 1:

Input:

World table:

name	continent	area	population	gdp
Afghanistan	Asia	652230	25500100	20343000000
Albania	Europe	28748	2831741	12960000000
Algeria	Africa	2381741	37100000	188681000000
Andorra	Europe	468	78115	3712000000
Angola	Africa	1246700	20609294	100990000000

Output:

name	population	area
Afghanistan	25500100	652230
Algeria	37100000	2381741

Solution :

</> Code

Pandas  Auto

```
1 import pandas as pd
2
3 def big_countries(world: pd.DataFrame) -> pd.DataFrame:
4     var= (world['area'] >= 3000000) | (world['population'] >= 25000000)
5     return world[var][['name' , 'population' , 'area']]
```

Q: 4

1148. Article Views I

Solve

Easy Topics Companies

[SQL Schema](#) > [Pandas Schema](#) >

Table: Views

Column Name	Type
article_id	int
author_id	int
viewer_id	int
view_date	date

There is no primary key (column with unique values) for this table, the table may have duplicate rows.
Each row of this table indicates that some viewer viewed an article (written by some author) on some date.
Note that equal author_id and viewer_id indicate the same person.

Write a solution to find all the authors that viewed at least one of their own articles.

Return the result table sorted by `id` in ascending order.

Example 1:

Input:

Views table:

article_id	author_id	viewer_id	view_date
1	3	5	2019-08-01
1	3	6	2019-08-02
2	7	7	2019-08-01
2	7	6	2019-08-02
4	7	1	2019-07-22
3	4	4	2019-07-21
3	4	4	2019-07-21

Output:

id
4
7

Solution :

```
</> Code
Pandas Auto
1 import pandas as pd
2
3 def article_views(views: pd.DataFrame) -> pd.DataFrame:
4     var = views['author_id'] == views['viewer_id']
5     return views[var][['author_id']].drop_duplicates().sort_values(by='author_id').rename(columns = {"author_id" : "id"})
6
```

Q:5

1683. Invalid Tweets

Solved ✓

Easy Topics Companies

SQL Schema > Pandas Schema >

Table: Tweets

Column Name	Type
tweet_id	int
content	varchar

tweet_id is the primary key (column with unique values) for this table.
content consists of alphanumeric characters, '!', or ' ' and no other special characters.
This table contains all the tweets in a social media app.

Write a solution to find the IDs of the invalid tweets. The tweet is invalid if the number of characters used in the content of the tweet is **strictly greater** than 15.

Return the result table in **any order**.

The result format is in the following example.

Example 1:

Input:

Tweets table:

tweet_id	content
1	Let us Code
2	More than fifteen chars are here!

Output:

tweet_id
2

Explanation:

Tweet 1 has length = 11. It is a valid tweet.

Tweet 2 has length = 33. It is an invalid tweet.

Solution :

</> Code

Pandas ▾ 🔒 Auto

```
1 import pandas as pd
2
3 def invalid_tweets(tweets: pd.DataFrame) -> pd.DataFrame:
4     var = tweets['content'].str.len() > 15
5     return tweets[var][['tweet_id']]
6
```

Q- 6

1378. Replace Employee ID With The Unique Identifier

Easy Topics Companies

[SQL Schema](#) > [Pandas Schema](#) >

Table: Employees

Column Name	Type
id	int
name	varchar

id is the primary key (column with unique values) for this table.

Each row of this table contains the id and the name of an employee in a company.

Table: EmployeeUNI

Column Name	Type
id	int
unique_id	int

(id, unique_id) is the primary key (combination of columns with unique values) for this table.

Each row of this table contains the id and the corresponding unique id of an employee in the company.

Write a solution to show the **unique ID** of each user, If a user does not have a unique ID replace just show `null`.

Return the result table in **any** order.

The result format is in the following example.

Example 1:

Input:

Employees table:

id	name
1	Alice
7	Bob
11	Meir
90	Winston
3	Jonathan

EmployeeUNI table:

id	unique_id
3	1
11	2
90	3

Output:

unique_id	name
null	Alice
null	Bob
2	Meir
3	Winston
1	Jonathan

Explanation:

Alice and Bob do not have a unique ID, We will show null instead.

The unique ID of Meir is 2.

The unique ID of Winston is 3.

The unique ID of Jonathan is 1.

Solution :**</> Code**

Pandas   Auto  

```
1 import pandas as pd
2
3 def replace_employee_id(employees: pd.DataFrame, employee_uni: pd.DataFrame) -> pd.DataFrame:
4     df_merge = pd.merge(left = employees , right = employee_uni , how ='left' , on = 'id')
5     return df_merge[['unique_id' , 'name']]
```

Q: 7

1068. Product Sales Analysis I

Sol

Easy Topics Companies

[SQL Schema](#) > [Pandas Schema](#) >

Table: Sales

Column Name	Type
sale_id	int
product_id	int
year	int
quantity	int
price	int

(sale_id, year) is the primary key (combination of columns with unique values) of this table.

product_id is a foreign key (reference column) to Product table.

Each row of this table shows a sale on the product product_id in a certain year.

Note that the price is per unit.

Table: Product

Column Name	Type
product_id	int
product_name	varchar

product_id is the primary key (column with unique values) of this table.

Each row of this table indicates the product name of each product.

Write a solution to report the `product_name`, `year`, and `price` for each `sale_id` in the `Sales` table.

Return the resulting table in **any order**.

The result format is in the following example.

Example 1:

Input:

Sales table:

sale_id	product_id	year	quantity	price
1	100	2008	10	5000
2	100	2009	12	5000
7	200	2011	15	9000

Product table:

product_id	product_name
100	Nokia
200	Apple
300	Samsung

Output:

product_name	year	price
Nokia	2008	5000
Nokia	2009	5000
Apple	2011	9000

Explanation:

From `sale_id = 1`, we can conclude that Nokia was sold for 5000 in the year 2008.

From `sale_id = 2`, we can conclude that Nokia was sold for 5000 in the year 2009.

From `sale_id = 7`, we can conclude that Apple was sold for 9000 in the year 2011.

Solution :

```
</> Cod Run Ctrl 
Pandas Auto
1 import pandas as pd
2
3 def sales_analysis(sales: pd.DataFrame, product: pd.DataFrame) -> pd.DataFrame:
4     df_merge = pd.merge(left=sales, right=product, how='inner', on='product_id')
5     return df_merge[['product_name', 'year', 'price']]
6
```

Q:8

1581. Customer Who Visited but Did Not Make Any Transactions

Solved 

Easy Topics Companies

[SQL Schema](#) > [Pandas Schema](#) >

Table: Visits

Column Name	Type
visit_id	int
customer_id	int

visit_id is the column with unique values for this table.

This table contains information about the customers who visited the mall.

Table: Transactions

Column Name	Type
transaction_id	int
visit_id	int
amount	int

transaction_id is column with unique values for this table.

This table contains information about the transactions made during the visit_id.

Write a solution to find the IDs of the users who visited without making any transactions and the number of times they made these types of visits.

Return the result table sorted in **any order**.

Example 1:

Input:

Visits

visit_id	customer_id
1	23
2	9
4	30
5	54
6	96
7	54
8	54

Transactions

transaction_id	visit_id	amount
2	5	310
3	5	300
9	5	200
12	1	910
13	2	970

Output:

customer_id	count_no_trans
54	2
30	1
96	1

Explanation:

Customer with id = 23 visited the mall once and made one transaction during the visit with id = 12.

Customer with id = 9 visited the mall once and made one transaction during the visit with id = 13.

Customer with id = 30 visited the mall once and did not make any transactions.

Customer with id = 54 visited the mall three times. During 2 visits they did not make any transactions, and during one visit they made 3 transactions.

Customer with id = 96 visited the mall once and did not make any transactions.

As we can see, users with IDs 30 and 96 visited the mall one time without making any transactions. Also, user 54 visited the mall twice and did not make any transactions.

Solution :

```
Pandas ▾ 🔒 Auto
1 import pandas as pd
2
3 def find_customers(visits: pd.DataFrame, transactions: pd.DataFrame) -> pd.DataFrame:
4     df_merge = pd.merge(left = visits ,right = transactions , how = 'left' , on = 'visit_id')
5     var = df_merge['transaction_id'].isnull()
6     return df_merge[var].groupby('customer_id').agg({'visit_id': 'count'}).reset_index().rename(columns = {"visit_id" : "count_no_trans"})
```

Q: 9

197. Rising Temperature

Easy Topics Companies

[SQL Schema](#) > [Pandas Schema](#) >

Table: Weather

Column Name	Type
id	int
recordDate	date
temperature	int

id is the column with unique values for this table.

There are no different rows with the same recordDate.

This table contains information about the temperature on a certain day.

Write a solution to find all dates' `id` with higher temperatures compared to its previous dates (yesterday).

Return the result table in **any order**.

The result format is in the following example.

Example 1:

Input:

Weather table:

id	recordDate	temperature
1	2015-01-01	10
2	2015-01-02	25
3	2015-01-03	20
4	2015-01-04	30

Output:

id
2
4

Explanation:

In 2015-01-02, the temperature was higher than the previous day (10 → 25).

In 2015-01-04, the temperature was higher than the previous day (20 → 30).

Solution :

```
Pandas  Auto  { } ↺ ↻ ↗

1 import pandas as pd
2
3 def rising_temperature(weather: pd.DataFrame) -> pd.DataFrame:
4     weather.sort_values(by='recordDate', inplace=True)
5     weather['Prev_temp'] = weather['temperature'].shift(1)
6     weather['Prev_recordDate'] = weather['recordDate'].shift(1)
7     weather['days_diff'] = (weather['recordDate'] - weather['Prev_recordDate']).dt.days
8
9
10    var = weather[(weather['temperature'] > weather['Prev_temp']) & (weather['days_diff'] == 1)]
11
12    return var[['id']]

Saved  Ln 1, Col 1
```

Q:10

1661. Average Time of Process per Machine

Solved

Easy Topics Companies

[SQL Schema](#) > [Pandas Schema](#) >

Table: Activity

Column Name	Type
machine_id	int
process_id	int
activity_type	enum
timestamp	float

The table shows the user activities for a factory website.

(machine_id, process_id, activity_type) is the primary key (combination of columns with unique values) of this table.

machine_id is the ID of a machine.

process_id is the ID of a process running on the machine with ID machine_id.

activity_type is an ENUM (category) of type ('start', 'end').

timestamp is a float representing the current time in seconds.

'start' means the machine starts the process at the given timestamp and 'end' means the machine ends the process at the given timestamp.

The 'start' timestamp will always be before the 'end' timestamp for every (machine_id, process_id) pair.

It is guaranteed that each (machine_id, process_id) pair has a 'start' and 'end' timestamp.

There is a factory website that has several machines each running the **same number of processes**. Write a solution to find the **average time** each machine takes to complete a process.

The time to complete a process is the 'end' timestamp minus the 'start' timestamp. The average time is calculated by the total time to complete every process on the machine divided by the number of processes that were run.

The resulting table should have the machine_id along with the **average time** as processing_time, which should be **rounded to 3 decimal places**.

Return the result table in **any order**.

The result format is in the following example.

Example 1:

Input:

Activity table:

machine_id	process_id	activity_type	timestamp
0	0	start	0.712
0	0	end	1.520
0	1	start	3.140
0	1	end	4.120
1	0	start	0.550
1	0	end	1.550
1	1	start	0.430
1	1	end	1.420
2	0	start	4.100
2	0	end	4.512
2	1	start	2.500
2	1	end	5.000

Output:

machine_id	processing_time
0	0.894
1	0.995
2	1.456

Explanation:

There are 3 machines running 2 processes each.

Machine 0's average time is $((1.520 - 0.712) + (4.120 - 3.140)) / 2 = 0.894$

Machine 1's average time is $((1.550 - 0.550) + (1.420 - 0.430)) / 2 = 0.995$

Machine 2's average time is $((4.512 - 4.100) + (5.000 - 2.500)) / 2 = 1.456$

Solution :**</> Code**

Pandas Auto

```
1 import pandas as pd
2
3 def get_average_time(activity: pd.DataFrame) -> pd.DataFrame:
4     df=activity
5     df_start = df[df['activity_type'] == 'start']
6     df_end = df[df['activity_type'] == 'end']
7
8     final_df = pd.merge(left = df_start , right = df_end ,how= 'inner', on =['machine_id' , 'process_id'] )
9
10    final_df['processing_time'] = final_df['timestamp_y'] - final_df['timestamp_x']
11    sol = final_df.groupby('machine_id').agg({"processing_time" : "mean"}).reset_index().round(3)
12
13    return sol
14
15    return final_df
16
```

Q: 11

577. Employee Bonus

Solved 

Easy  Topics  Companies  Hint

[SQL Schema](#) > [Pandas Schema](#) >

Table: `Employee`

Column Name	Type
empId	int
name	varchar
supervisor	int
salary	int

empId is the column with unique values for this table.

Each row of this table indicates the name and the ID of an employee in addition to their salary and the id of their manager.

Table: `Bonus`

Column Name	Type
empId	int
bonus	int

empId is the column of unique values for this table.

empId is a foreign key (reference column) to empId from the Employee table.

Each row of this table contains the id of an employee and their respective bonus.

Write a solution to report the name and bonus amount of each employee with a bonus **less than** 1000.

Return the result table in **any order**.

The result format is in the following example.

Example 1:

Input:

Employee table:

empId	name	supervisor	salary
3	Brad	null	4000
1	John	3	1000
2	Dan	3	2000
4	Thomas	3	4000

Bonus table:

empId	bonus
2	500
4	2000

Output:

name	bonus
Brad	null
John	null
Dan	500

Solution :

</> Code

Pandas Auto

```
1 import pandas as pd
2
3 def employee_bonus(employee: pd.DataFrame, bonus: pd.DataFrame) -> pd.DataFrame:
4     df_merge = pd.merge(left = employee , right = bonus , how = 'left' , on = 'empId')
5     sol = df_merge[(df_merge['bonus'] <= 1000) | (df_merge['bonus'].isnull())]
6     #sol = df_merge.query("bonus <= 1000 or bonus.isnull()")
7
8     return sol[['name' , 'bonus']]
9
```

Q:12

1280. Students and Examinations

Solved 

Easy  Topics  Companies

[SQL Schema](#) > [Pandas Schema](#) >

Table: `Students`

Column Name	Type
<code>student_id</code>	<code>int</code>
<code>student_name</code>	<code>varchar</code>

`student_id` is the primary key (column with unique values) for this table.
Each row of this table contains the ID and the name of one student in the school.

Table: `Subjects`

Column Name	Type
<code>subject_name</code>	<code>varchar</code>

`subject_name` is the primary key (column with unique values) for this table.
Each row of this table contains the name of one subject in the school.

Table: `Examinations`

Column Name	Type
<code>student_id</code>	<code>int</code>
<code>subject_name</code>	<code>varchar</code>

There is no primary key (column with unique values) for this table. It may contain duplicates.
Each student from the `Students` table takes every course from the `Subjects` table.
Each row of this table indicates that a student with ID `student_id` attended the exam of `subject_name`.

Write a solution to find the number of times each student attended each exam.

Return the result table ordered by `student_id` and `subject_name`.

The result format is in the following example.

Example 1:

Input:

Students table:

student_id	student_name
1	Alice
2	Bob
13	John
6	Alex

Subjects table:

subject_name
Math
Physics
Programming

Examinations table:

student_id	subject_name
1	Math
1	Physics
1	Programming
2	Programming
1	Physics
1	Math
13	Math
13	Programming
13	Physics
2	Math
1	Math

Output:

student_id	student_name	subject_name	attended_exams
1	Alice	Math	3
1	Alice	Physics	2
1	Alice	Programming	1
2	Bob	Math	1
2	Bob	Physics	0
2	Bob	Programming	1
6	Alex	Math	0
6	Alex	Physics	0
6	Alex	Programming	0
13	John	Math	1
13	John	Physics	1
13	John	Programming	1

Explanation:

The result table should contain all students and all subjects.

Alice attended the Math exam 3 times, the Physics exam 2 times, and the Programming exam 1 time.

Bob attended the Math exam 1 time, the Programming exam 1 time, and did not attend the Physics exam.

Alex did not attend any exams.

John attended the Math exam 1 time, the Physics exam 1 time, and the Programming exam 1 time.

Solution :

```
1 import pandas as pd
2
3 def students_and_examinations(students: pd.DataFrame, subjects: pd.DataFrame, examinations: pd.DataFrame) -> pd.DataFrame:
4     examinations['dummy'] = 1
5     df= pd.merge(left = students , right = subjects , how ='cross')
6     df_final = pd.merge(left = df , right = examinations , how = 'left' , on =['student_id' , 'subject_name'] )
7     return df_final.groupby(['student_id' , 'student_name' , 'subject_name'], dropna =False).agg({"dummy" : "count"}).
    reset_index().rename(columns ={"dummy" : "attended_exams"})
```

Q: 13

570. Managers with at Least 5 Direct Reports

Solved ✓

Medium 🔖 Topics 🏢 Companies 💡 Hint

[SQL Schema](#) > [Pandas Schema](#) >

Table: Employee

Column Name	Type
id	int
name	varchar
department	varchar
managerId	int

id is the primary key (column with unique values) for this table.

Each row of this table indicates the name of an employee, their department, and the id of their manager.

If managerId is null, then the employee does not have a manager.

No employee will be the manager of themselves.

Write a solution to find managers with at least **five direct reports**.

Return the result table in **any order**.

The result format is in the following example.

Example 1:

Input:

Employee table:

id	name	department	managerId
101	John	A	null
102	Dan	A	101
103	James	A	101
104	Amy	A	101
105	Anne	A	101
106	Ron	B	101

Output:

name
John

Solution :

 Code

Pandas  Auto   

```
1 import pandas as pd
2
3 def find_managers(employee: pd.DataFrame) -> pd.DataFrame:
4     managers = employee.groupby(
5         'managerId', as_index=False
6     ).agg(
7         reporting=('id', 'count'),
8     ).query(
9         '5 <= reporting'
10    )['managerId']
11
12    return employee[
13        employee['id'].isin(managers)
14    ][['name']]
15
```

Q: 14

1934. Confirmation Rate

Solved 

Medium

 Topics

 Companies

[SQL Schema](#) > [Pandas Schema](#) >

Table: Signups

Column Name	Type
user_id	int
time_stamp	datetime

user_id is the column of unique values for this table.

Each row contains information about the signup time for the user with ID user_id.

Table: Confirmations

Column Name	Type
user_id	int
time_stamp	datetime
action	ENUM

(user_id, time_stamp) is the primary key (combination of columns with unique values) for this table.

(user_id, time_stamp) is the primary key (combination of columns with unique values) for this table.

user_id is a foreign key (reference column) to the Signups table.

action is an ENUM (category) of the type ('confirmed', 'timeout')

Each row of this table indicates that the user with ID user_id requested a confirmation message at time_stamp and that confirmation message was either confirmed ('confirmed') or expired without confirming ('timeout').

The **confirmation rate** of a user is the number of 'confirmed' messages divided by the total number of requested confirmation messages. The confirmation rate of a user that did not request any confirmation messages is 0. Round the confirmation rate to **two decimal** places.

Write a solution to find the **confirmation rate** of each user.

Return the result table in **any order**.

The result format is in the following example.

Example 1:**Input:**

Signups table:

user_id	time_stamp
3	2020-03-21 10:16:13
7	2020-01-04 13:57:59
2	2020-07-29 23:09:44
6	2020-12-09 10:39:37

Confirmations table:

user_id	time_stamp	action
3	2021-01-06 03:30:46	timeout
3	2021-07-14 14:00:00	timeout
7	2021-06-12 11:57:29	confirmed
7	2021-06-13 12:58:28	confirmed
7	2021-06-14 13:59:27	confirmed
2	2021-01-22 00:00:00	confirmed
2	2021-02-28 23:59:59	timeout

Output:

user_id	confirmation_rate
6	0.00
3	0.00
7	1.00
2	0.50

Explanation:

User 6 did not request any confirmation messages. The confirmation rate is 0.

User 3 made 2 requests and both timed out. The confirmation rate is 0.

User 7 made 3 requests and all were confirmed. The confirmation rate is 1.

User 2 made 2 requests where one was confirmed and the other timed out. The confirmation rate is $1 / 2 = 0.5$.**Solution :**

```
1 import pandas as pd
2
3 def confirmation_rate(signups: pd.DataFrame, confirmations: pd.DataFrame) -> pd.DataFrame:
4     confirmations['confirmation_rate'] = confirmations['action'].apply(lambda x: 1 if x == 'confirmed' else 0)
5     avg_conf = confirmations[['user_id', 'confirmation_rate']].groupby('user_id')['confirmation_rate'].mean().round(2).
6     reset_index()
7     output = pd.merge(signups['user_id'], avg_conf, how='left').fillna(0)
8     return output
```